

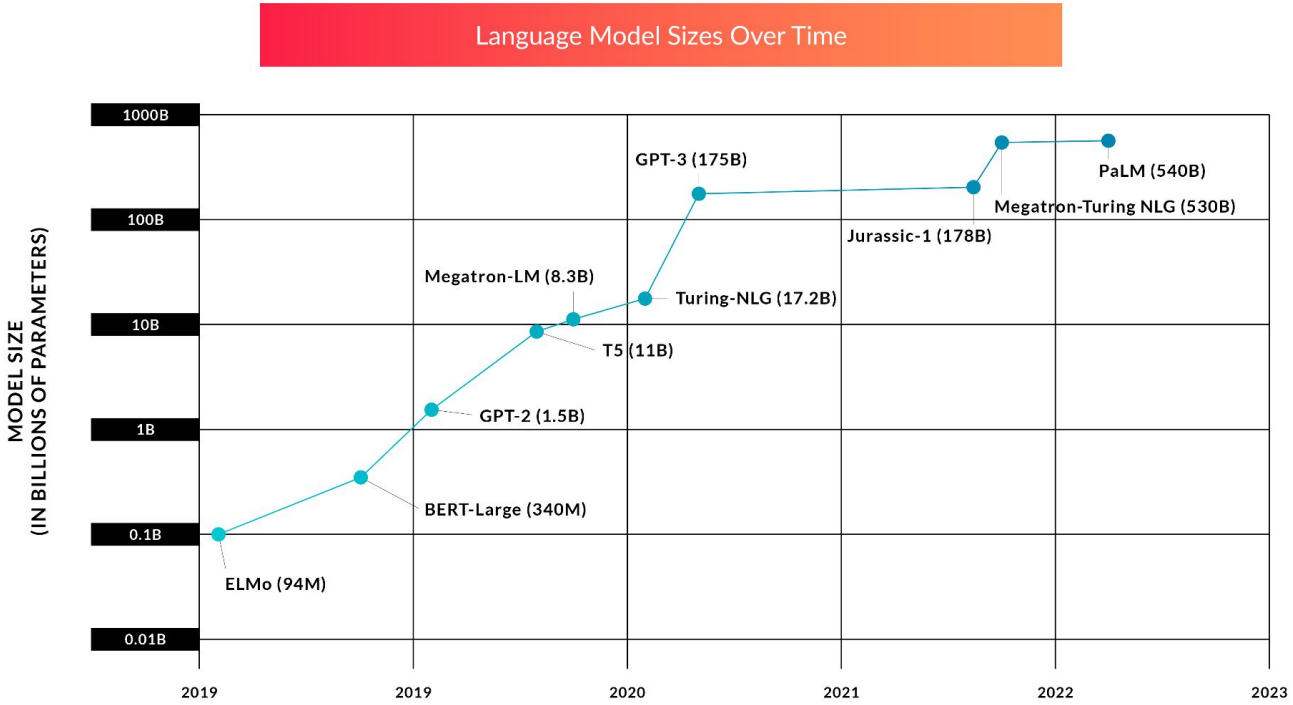
Квантизация и спарсификация

Введение в Эффективные Системы Глубокого Обучения

Феоктистов Дмитрий Дмитриевич, Yandex Research, BRAIn Lab {@trandelik}

Лекция 05, осень 2026

Мотивация


Language Model Sizes Over Time
MODEL SIZE
(IN BILLIONS OF PARAMETERS)
1000B
100B
10B
1B
0.1B
0.01B
2019
2019
2020
2021
2022
2023
ELMo (94M)
BERT-Large (340M)
GPT-2 (1.5B)
T5 (11B)
Megatron-LM (8.3B)
Turing-NLG (17.2B)
GPT-3 (175B)
Jurassic-1 (178B)
Megatron-Turing NLG (530B)
PaLM (540B)

План лекции

1. Спарсификация: важность весов и компенсация ошибки в OBS.
2. Квантизация: форматы, гранулярность, PTQ и QAT.
3. Обучение в FP8: смешанная точность на примере DeepSeek-V3.
4. От OBS к квантизации: постановка OBQ и вывод GPTQ.
5. LLM: выбросы, масштабирование, вращения и кодовые книги.
6. Квантизация и pruning: сравнение и сочетание.

Материалы занятия

- Лекции Егора Швецов

  https://github.com/On-Point-RND/Efficient-AI-Models-2024/blob/main/Week%202/Lectures/Pruning.pdf

  https://github.com/On-Point-RND/Efficient-AI-Models-2024/blob/main/Week%202/Lectures/Quatization.pdf

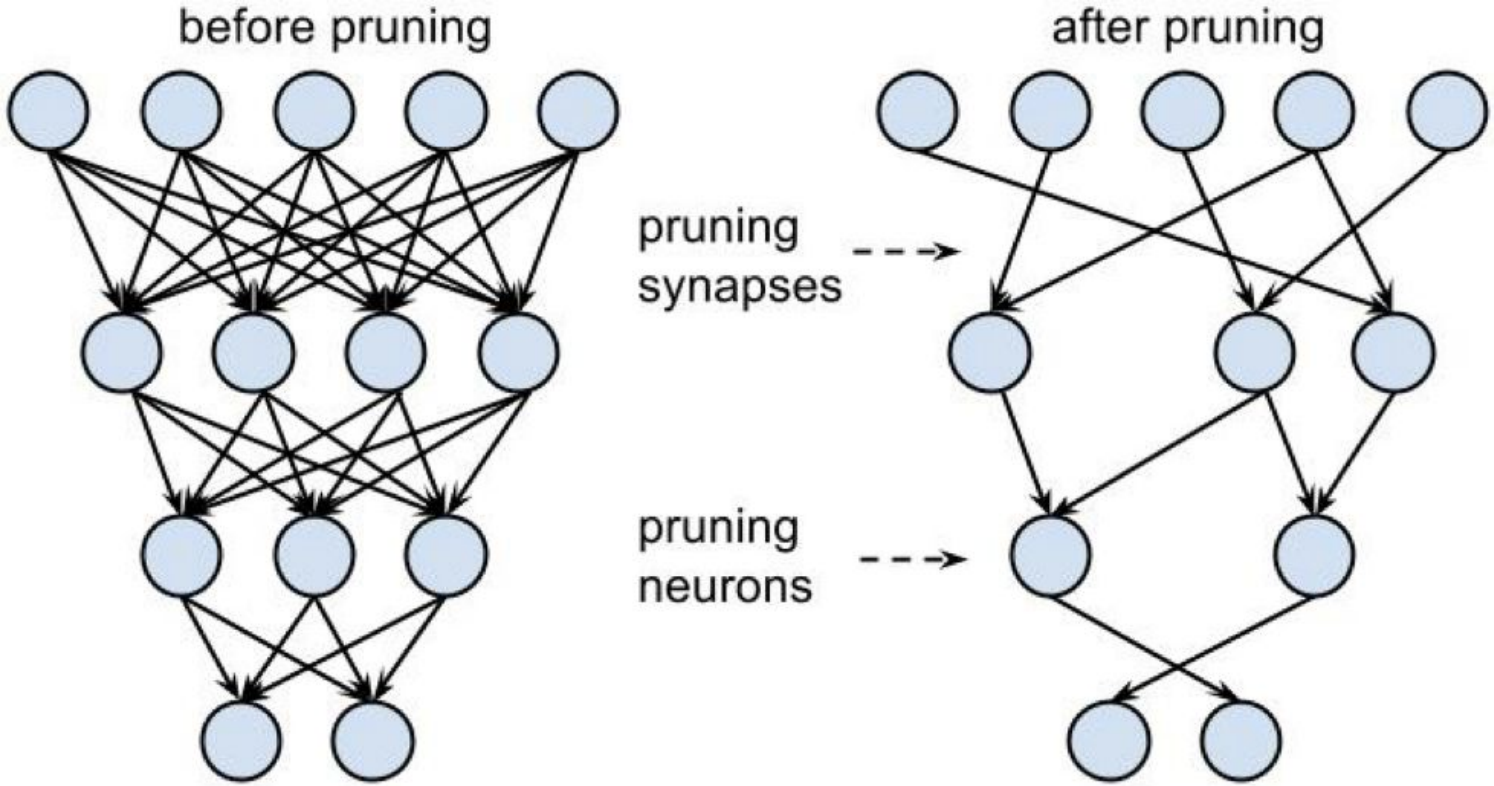
Спарсификация

- Заменяем часть весов нулями или удаляем целые структуры.
- Выбираем маску, затем при необходимости восстанавливаем качество.
- Различаем **неструктурную**, **N:M** и **структурную** разреженность.

$$\widehat{W} = M \odot W, \qquad M_{ij} \in \{0, 1\}.$$

Маска нулей сама по себе не меняет размер плотной матрицы.

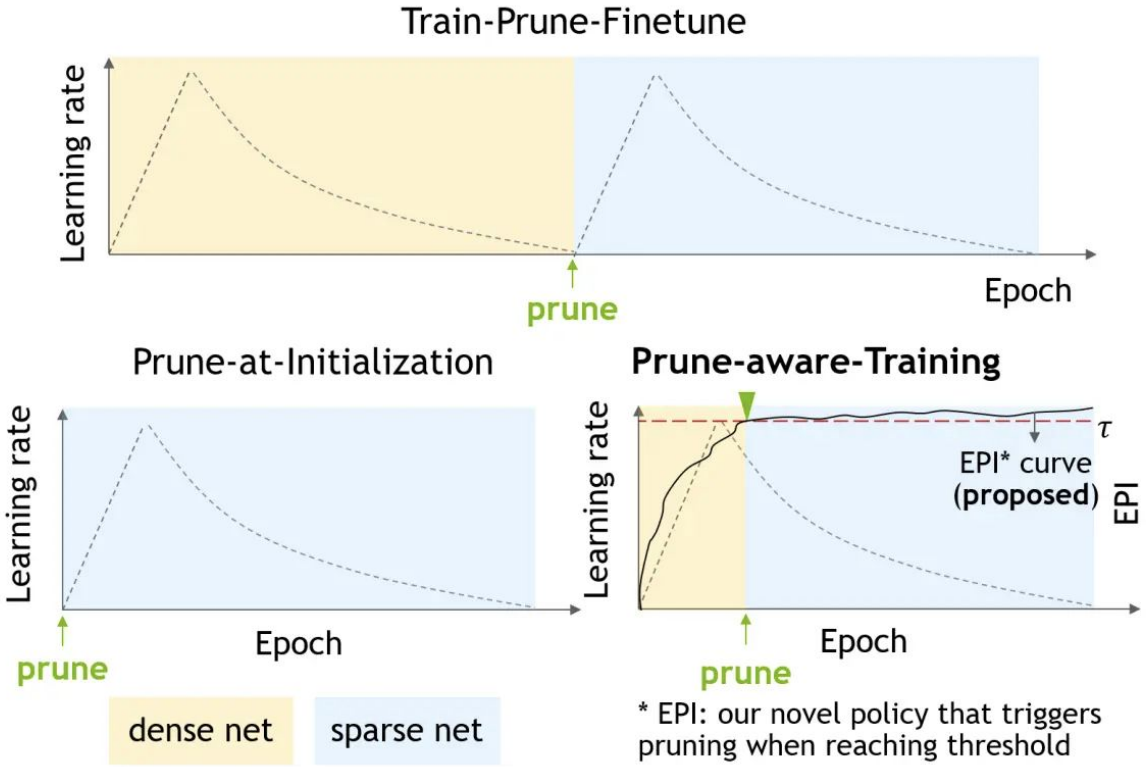
Общая идея


before pruning
after pruning
pruning
synapses
pruning
neurons

Общая идея

It is well documented that neural networks have an excess of parameters needed to generalize well and make accurate predictions. *“The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks” (Frankle and Carbin, 2019)* demonstrates that **neural networks tend to have a specific subset of parameters that are essential for prediction**. Model pruning is a very intuitive approach to model compression, with the hypothesis that effective model compression should **remove weights that aren’t being used**, similarly to how the brain reduces usage of connections between neurons to emphasize important pathways.

TPF, PAI, PTP, PAT



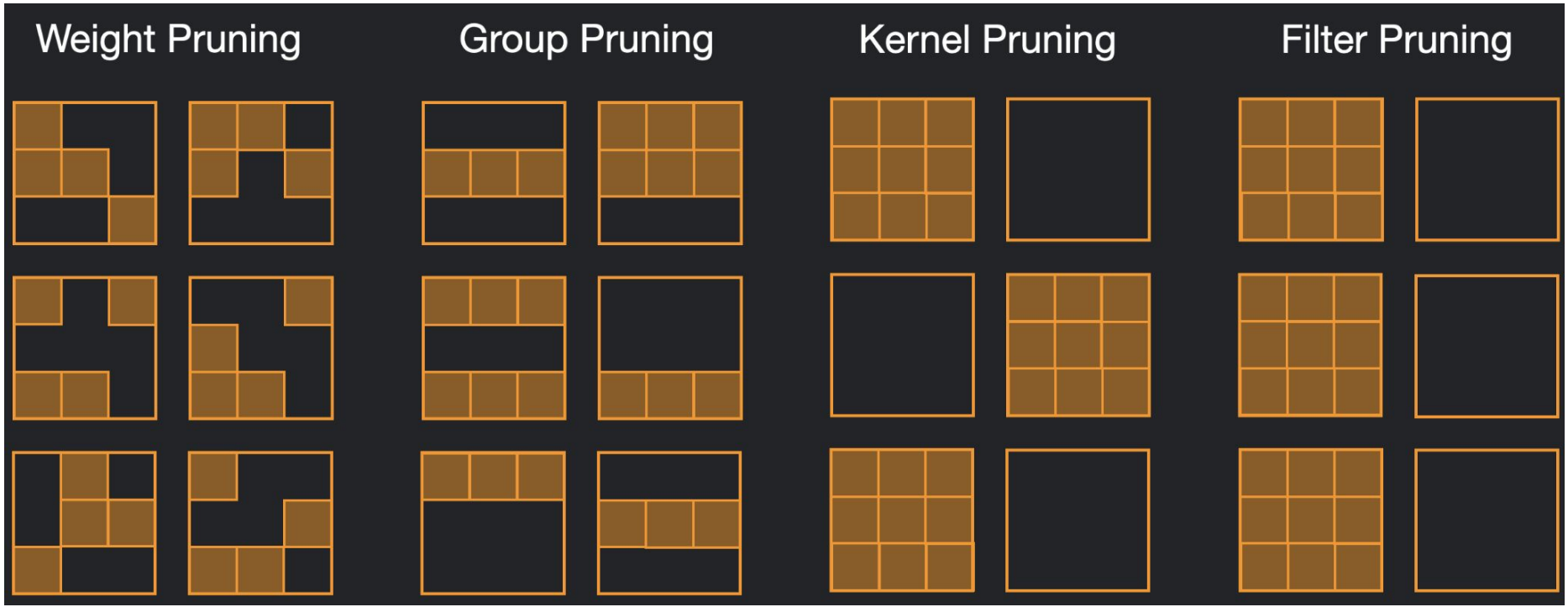
TPF, PAI, PTP, PAT

Datature | Blog

PRUNING APPROACH	TRAIN-TIME PRUNING	POST-TRAINING PRUNING
PROS	More efficient models since they are trained with the objective of sparsity in mind	Simpler to implement
	Pruning decisions are made alongside the consideration of model parameter optimization	Pruning parameters can be easily adjusted based on inference requirements
CONS	Makes training process more complex	May require fine-tuning to regain performance in the case of accuracy degradation
	Change in pruning parameters may require retraining of the whole model	Pruning decisions are user-defined and may not be optimal

TRADEOFFS OF DIFFERENT PRUNING APPROACHES

Structured vs Unstructured



Structured vs Unstructured

Input Operands	Accumulator	Dense TOPS	vs. FFMA	Sparse TOPS	vs. FFMA
FP32	FP32	19.5	–	–	–
TF32	FP32	156	8X	**312**	**16X**
FP16	FP32	312	16X	**624**	**32X**
BF16	FP32	312	16X	**624**	**32X**
FP16	FP16	312	16X	**624**	**32X**
INT8	INT32	624	32X	**1248**	**64X**

Table 1. Performance of Sparse Tensor Cores in the NVIDIA Ampere Architecture.

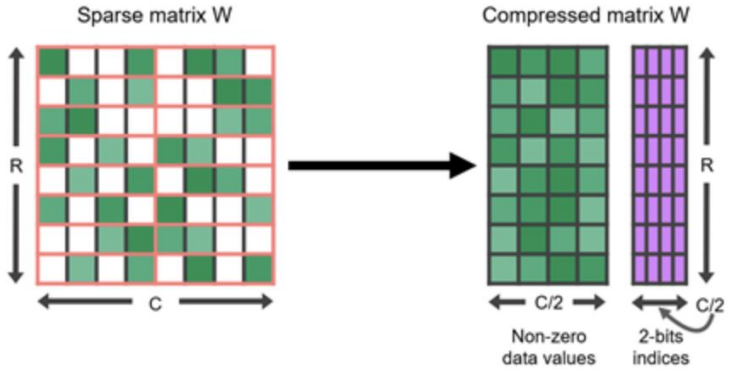
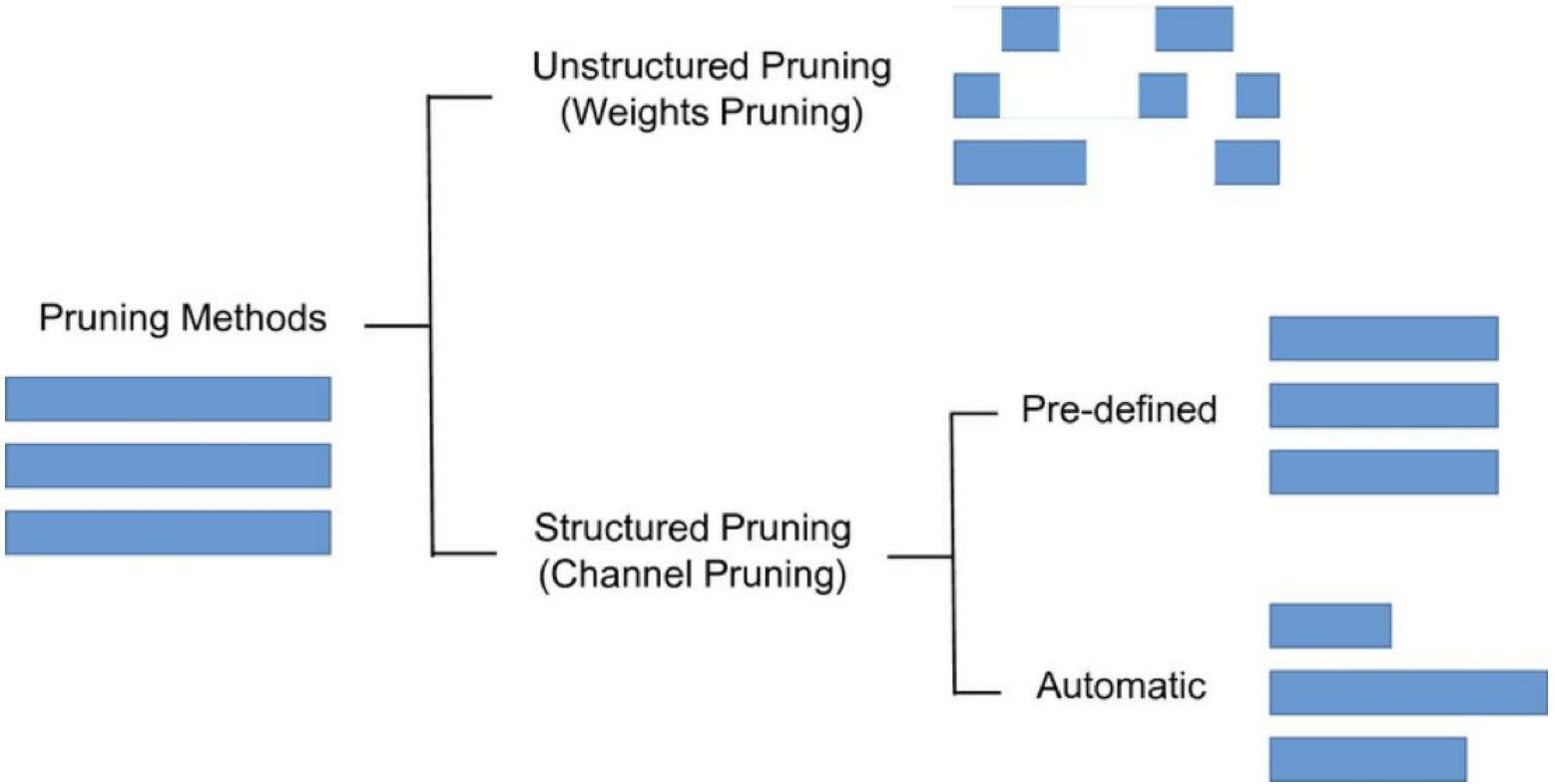


Figure 1. A 2:4 structured sparse matrix W, and its compressed representation

Structured vs Unstructured



Wanda: важен вес и его вход

Для линейного слоя $Y = WX$, $X \in \mathbb{R}^{d \times n}$:

$$\boxed{s_{ij} = |W_{ij}| \, \|X_{j,:}\|_2}$$

- Большой вес важен, если соответствующий вход активен.
- В каждой выходной строке удаляем веса с малым s_{ij}.
- Нужны калибровочные активации; градиенты не нужны.
- Для N:M выбираем маску внутри допустимых групп.

Wanda обнуляет выбранные веса. Далее OBS покажет, как изменить оставшиеся веса, чтобы компенсировать ошибку.

Sun et al. A Simple and Effective Pruning Approach for Large Language Models (Wanda), ICLR 2024.

OBS: удалить вес, компенсировать ошибку

Рядом со стационарной точкой w аппроксимируем потерю квадратично:

$$\mathcal{L}(w+\delta) - \mathcal{L}(w) \approx \tfrac{1}{2}\delta^\top H \delta, \qquad \nabla\mathcal{L}(w) \approx 0.$$

При удалении веса p нужно $w_p + \delta_p = 0$; e_p – базисный вектор:

$$\min_{\delta} \ \tfrac{1}{2}\delta^\top H \delta \quad \text{при} \quad e_p^\top \delta = -w_p.$$

- Остальные веса разрешено изменить, чтобы компенсировать удаление.
- Взаимодействия весов закодированы в H, а не только в $|w_p|$.
- Для вывода считаем $H \succ 0$; на практике нужна регуляризация.

Hassibi, Stork. Second Order Derivatives for Network Pruning: Optimal Brain Surgeon, NIPS 1992.

OBS: вывод через множитель Лагранжа

Обозначим $C = H^{-1}$. Для ограничения $e_p^\top \delta = -w_p$:

$$\mathcal{J}(\delta, \nu) = \tfrac{1}{2}\delta^\top H \delta + \nu(e_p^\top \delta + w_p).$$

$$H\delta + \nu e_p = 0 \implies \delta = -\nu C e_p, \qquad -\nu C_{pp} = -w_p.$$

Поэтому

$$\boxed{\delta^* = -\frac{w_p}{C_{pp}} C_{:,p}}, \qquad \boxed{\Delta\mathcal{L}_p \approx \frac{w_p^2}{2C_{pp}}}.$$

Выбор веса: удаляем p с минимальным прогнозируемым ростом потери и применяем δ^* ко всем свободным весам.

OBS, Hassibi & Stork, 1992. Формулы выведены здесь для выбранных обозначений.

Важность можно оценивать для группы весов

- Единицей удаления может быть отдельный вес или **группа**: канал, голова attention, блок.
- Задаём важность $S(G)$ группы G и удаляем наименее важные группы до нужного размера модели.
- Например, простой критерий – сумма квадратов весов: $S(G) = \Sigma_{j \in G} w_j^2$; точнее можно оценивать изменение потери.
- При удалении канала согласованно удаляем связанные строки и столбцы соседних матриц.

Идея та же: выбрать, что удалить, затем при необходимости восстановить качество.
Теперь важность сопоставляется целой группе.

Квантизация

- Ограничиваем множество допустимых значений весов / активаций.
- Выбираем тип, масштаб и гранулярность.
- PTQ: преобразуем обученную модель.
- QAT: учитываем ошибку квантизации при обучении.

Связь с pruning: в OBS фиксировали вес в нуле. После основ квантизации обобщим этот шаг на фиксацию веса в точке сетки – получим OBQ.

Типы данных и поколения NVIDIA GPU

Тип	Память*	Пик к FP16†	Поколение / пример GPU
FP16	$2\times$	$1\times$	Volta / V100
BF16	$2\times$	$1\times$	Ampere / A100
INT8	$4\times$	$2\times$	Turing / T4
INT4	$8\times$	$4\times$	Turing / T4
INT1 / binary	$32\times$	–	Turing / T4
FP8	$4\times$	$2\times$	Ada / L40; Hopper / H100
FP4 (E2M1)	$8\times$	$4\times$	Blackwell / B200

Появление операций Tensor Core на дискретных NVIDIA GPU; оговорки – далее.

*Сокращение объёма самих значений относительно FP32; scales и другая метаинформация не учтены.

†Примеры отношений пиков *dense* Tensor Core throughput на одной GPU: INT8/INT4 – Turing, FP8 – H100, FP4 – B200. Это не ускорение всей модели.

NVIDIA: PTX ISA; Turing; Ada; B200.

Что означает «поддерживается на GPU»

- **Хранение $\neq$ матмул.** W4A16 может хранить INT4-веса, распаковывая их в FP16/BF16 внутри ядра.
- **Аккумулятор шире входов.** Например, INT8 $\times$ INT8 $\rightarrow$ INT32; FP16 $\times$ FP16 $\rightarrow$ FP32.
- **INT4 / INT1 – не монотонное «с поколения и выше».** В Hopper удалены прежние WMMA-операции INT4 и binary XOR; поддержка зависит от инструкции и API.
- Для binary используются логическая операция и popcount; это не обычный INT1-матмул с привычной арифметикой.

FP16 на CUDA-ядрах появился до Volta. В таблице указано появление именно Tensor Core-операций; программная поддержка проверяется отдельно.

CUDA C++ Programming Guide: Warp Matrix Functions, Sub-byte Operations.

Зачем целые, а зачем плавающая точка?

INT: равномерная сетка

$$\hat{x} = s(q - z)$$

- Равномерная сетка и простая целочисленная арифметика.
- Хорошая абсолютная точность в выбранном диапазоне.
- Выбросы растягивают диапазон: растёт шаг для остальных значений.

FP: экспонента + мантисса

$$\hat{x} = s \cdot (-1)^b 2^e (1 + m)$$

- У нормальных чисел шаг растёт с величиной.
- Экспонента расширяет диапазон; мантисса задаёт точность.
- Меньше уровней на узком интервале при том же числе бит.

Ни один тип не «точнее вообще». Выбираем сетку под распределение, допустимую ошибку и доступные операции. У FP-форматов тоже есть масштабы и clipping.

FP8: E4M3 и E5M2

$$\underbrace{1}_{\text{знак}} + \underbrace{4}_{\text{экспонента}} + \underbrace{3}_{\text{мантисса}} = 8 \qquad 1 + 5 + 2 = 8$$

- **E4M3**: больше точности, меньше диапазон; максимум 448.
- **E5M2**: шире диапазон, меньше точности; максимум 57344.
- Типичный выбор: E4M3 для весов/активаций; E5M2 полезен для градиентов с большим диапазоном.
- Масштабирование всё равно нужно: $q = \mathrm{cast}_{\mathrm{FP8}}(x/s)$, $\hat{x} = sq$.

В режиме W8A8 оба входа матмула представлены в FP8; тип аккумуляции задаётся отдельно.

NVIDIA Transformer Engine: FP8 Current Scaling.

FP4 и микромасштабирование

FP4 E2M1: знак + 2 бита экспоненты + 1 бит мантиссы.

Неотрицательные уровни: $0,\ 0.5,\ 1,\ 1.5,\ 2,\ 3,\ 4,\ 6$. Нужен масштаб.

Схема	Значения	Блок	Масштаб
MXFP8	FP8	32	E8M0: степень двойки
MXFP4	FP4 E2M1	32	E8M0: степень двойки
NVFP4	FP4 E2M1	16	E4M3 на блок + FP32 на тензор

- Меньший блок адаптируется к локальному диапазону.
- Цена – память под scales: MXFP4 $= 4 + 8/32 = 4.25$ бит/значение; NVFP4 $\approx 4 + 8/16 = 4.5$ без общего scale.
- Масштабирование – часть схемы квантизации.

OCP Microscaling Formats (MX) v1.0; NVIDIA: Introducing NVFP4; TensorRT quantization schemes.

Равномерная квантизация: scale и zero-point

Пусть $[a, b]$ – выбранный диапазон с $a \le 0 \le b$, $a < b$, а $[q_{\min}, q_{\max}]$ – целочисленные коды.

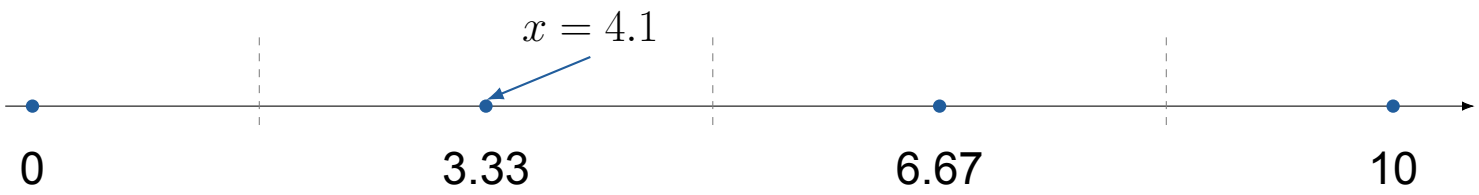
$$s = \frac{b-a}{q_{\max} - q_{\min}}, \qquad z = \operatorname{clip}\left(\operatorname{round}\left(q_{\min} - \frac{a}{s}\right), q_{\min}, q_{\max}\right).$$

$$q = \operatorname{clip}(\operatorname{round}(x/s) + z, q_{\min}, q_{\max}), \qquad \boxed{\hat{x} = s(q - z)}.$$

- z – код нуля; s – шаг между соседними значениями.
- Clipping ограничивает диапазон; rounding создаёт ошибку внутри него.
- Симметричная схема: $z = 0$, например $q \in [-127, 127]$ и $s = \max|x|/127$ для INT8.

Пример: диапазон $[0, 10]$, 2 бита

$$q \in \{0, 1, 2, 3\}, \quad s = 10/3, \quad z = 0, \qquad \hat{x} \in \{0, 3.33, 6.67, 10\}.$$



Здесь весь положительный диапазон использован. Если взять signed-коды и забыть zero-point, верхняя часть диапазона может обрезаться.

Lloyd–Max: как выбрать неравномерную сетку

Для $K = 2^b$ уровней c_i ищем границы интервалов t_i, минимизирующие среднюю квадратичную ошибку:

$$\min_{\{c_i, t_i\}} \sum_{i=1}^{K} \int_{t_{i-1}}^{t_i} (x - c_i)^2 p(x)\, dx, \qquad t_0 = -\infty, \quad t_K = +\infty.$$

1. **Фиксируем границы:** каждый уровень – среднее значений своего интервала: $c_i = \mathbb{E}[x \mid t_{i-1} \le x < t_i]$.
2. **Фиксируем уровни:** граница посередине между соседними уровнями: $t_i = (c_i + c_{i+1})/2$.
3. Начинаем с выбранной сетки и чередуем два шага до сходимости.

Это скалярный аналог k-means: ошибка не растёт, но глобальный минимум не гарантирован. Интервалы обычно имеют разную ширину и разную вероятность.

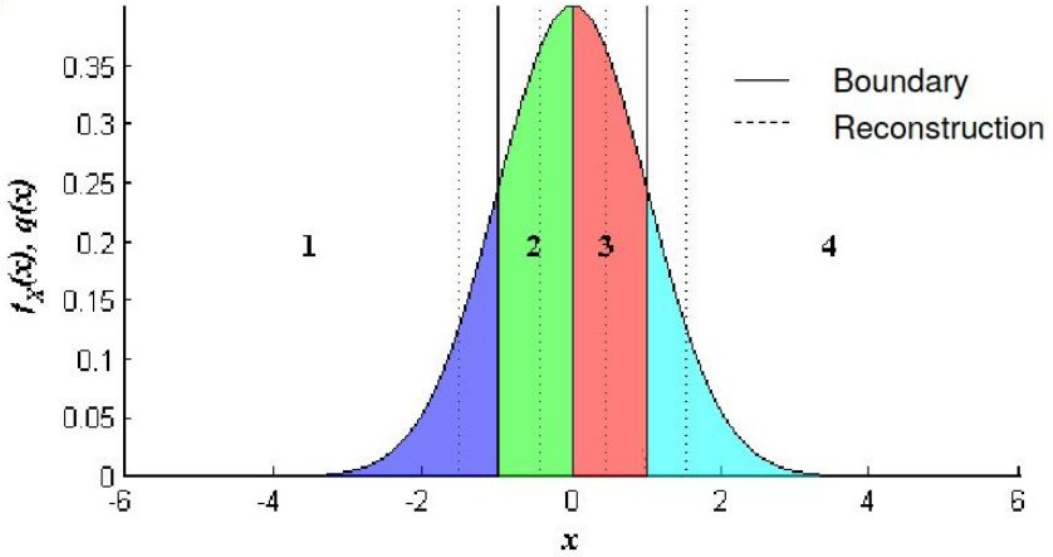
MIT OCW, Principles of Digital Communications I: Quantization, §3.2.3, Lloyd–Max algorithm.

Неравномерная квантизация: Lloyd–Max

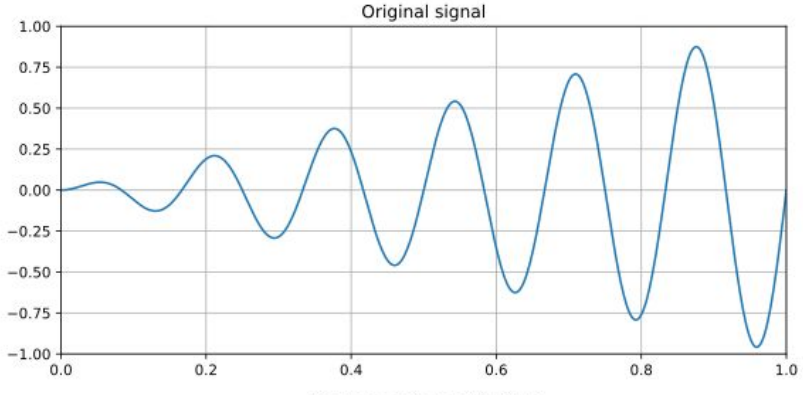
Decision thresholds -0.98, 0, 0.98

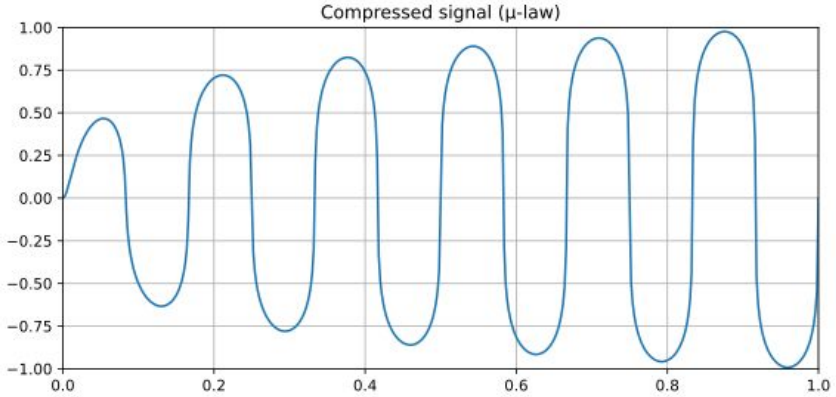
Representative levels –1.51, -0.45, 0.45, 1.51

D^*=0.12=9.30 dB



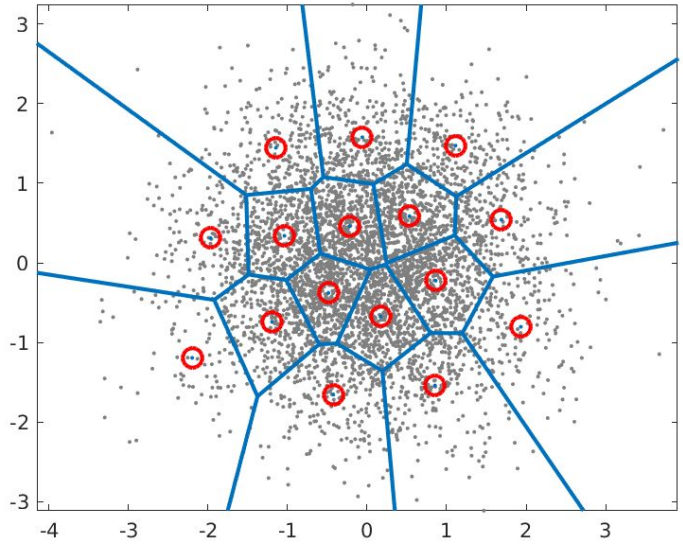
Квантизация: companding

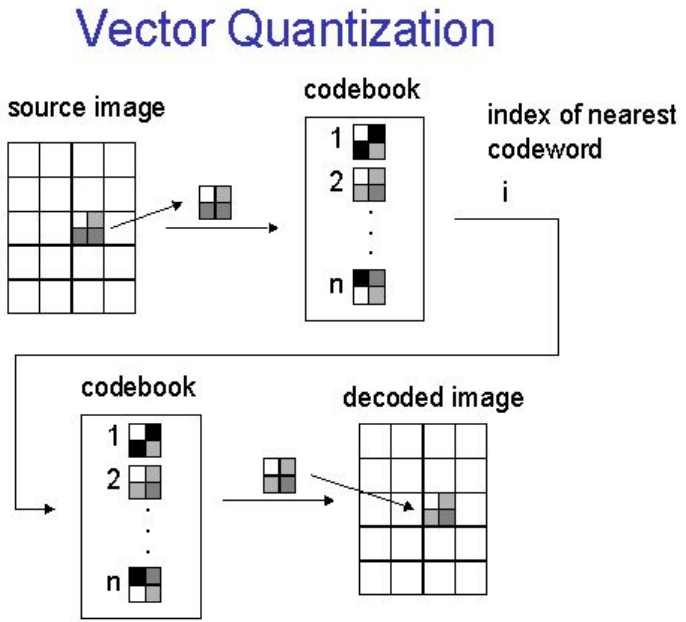




Преобразование должно быть дешевым, мы все же хотим быстрый инференс

Квантизация: векторная





\+ иерархичность

Что квантизовать?

- Веса
- Веса + активации
- KV-cache

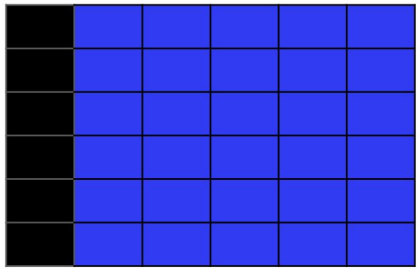
Что квантизовать?

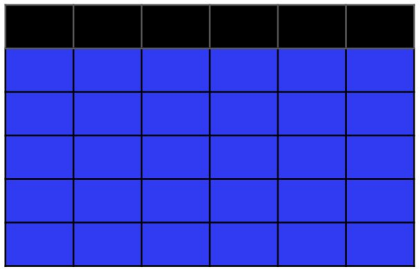
- **Веса**: например, W4A16 – 4-битные веса, 16-битные активации.
- **Веса и активации**: W8A8, W4A4; нужны соответствующие вычислительные ядра.
- **KV-cache**: отдельный объект квантизации, независимый от битности весов.

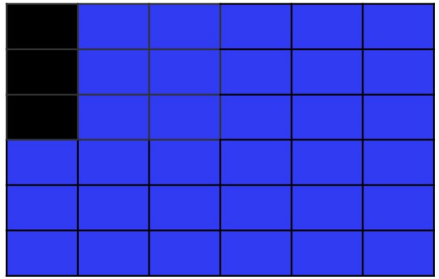
Формат хранения, формат операций и тип аккумуляции – три разных решения.

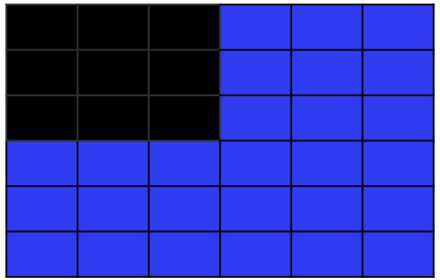
Компактное представление уменьшает объём данных. Чтобы использовать его в вычислениях, нужна поддержка в выбранном backend.

Гранулярность





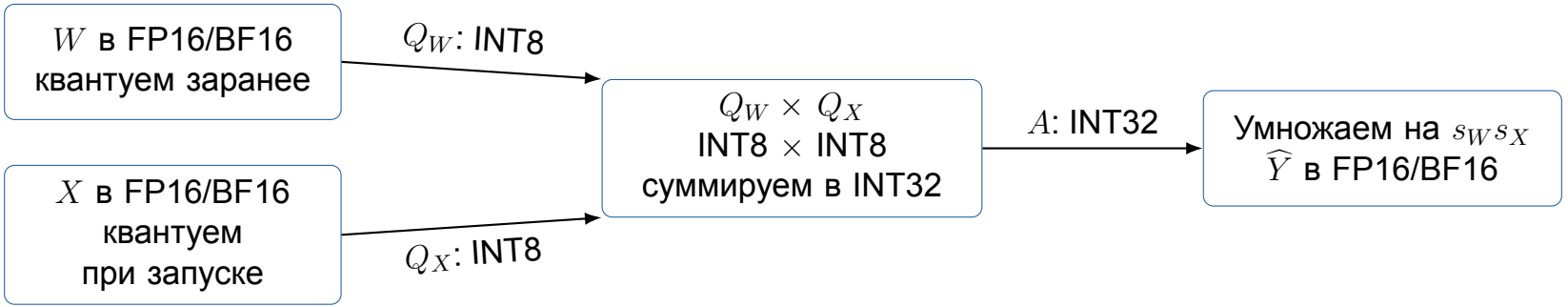




W8A8: откуда берётся matmul в целых числах

Симметричная INT8-квантизация ($z_W = z_X = 0$), один масштаб на тензор:

$$W \approx s_W Q_W, \quad X \approx s_X Q_X \quad \Longrightarrow \quad WX \approx s_W s_X \underbrace{(Q_W Q_X)}_{\text{целочисленный matmul}} .$$



$$A_{ij} = \sum_k (Q_W)_{ik} (Q_X)_{kj}, \qquad \widehat{Y}_{ij} = s_W s_X A_{ij}.$$

Масштабы вынесли за сумму: они не зависят от k. Для групп вдоль k масштабируем частичные суммы отдельно; при $z \neq 0$ нужны поправки на zero-point.

Jacob et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, CVPR 2018.

PTQ и QAT

PTQ: после обучения

1. Берём готовую модель.
2. Подбираем сетку; при необходимости используем калибровочные данные.
3. Квантуем и сохраняем веса.

Возможны коррекция ошибки и локальная реконструкция слоёв. Полное переобучение не требуется.

QAT: учитываем квантизацию

1. Добавляем fake quantization в forward.
2. Обучаем или **дообучаем готовую модель**.
3. Экспортируем в целевой формат.

Обычно сохраняем вещественные master weights; для backward используем оценку градиента.

QAT не требует обучения с нуля и сам по себе не означает, что все операции обучения выполняются в низкой точности.

Jacob et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, CVPR 2018.

QAT: straight-through estimator (STE)

$$\widehat{W} = \mathrm{dequant}(\mathrm{quant}(W)), \qquad \mathcal{L} = \ell(F(x; \widehat{W}), y).$$

Округление имеет нулевую производную почти всюду. В backward подставляем приближение:

$$\frac{\partial \widehat{W}}{\partial W} \approx I \quad \Longrightarrow \quad W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial \widehat{W}}.$$

- Обновляются вещественные master weights W.
- Возможен clipped STE: пропускаем градиент только внутри диапазона.
- STE – смещённая оценка, а не точная производная округления.

Bengio, Léonard, Courville. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation, 2013.

QAT: DiffQ

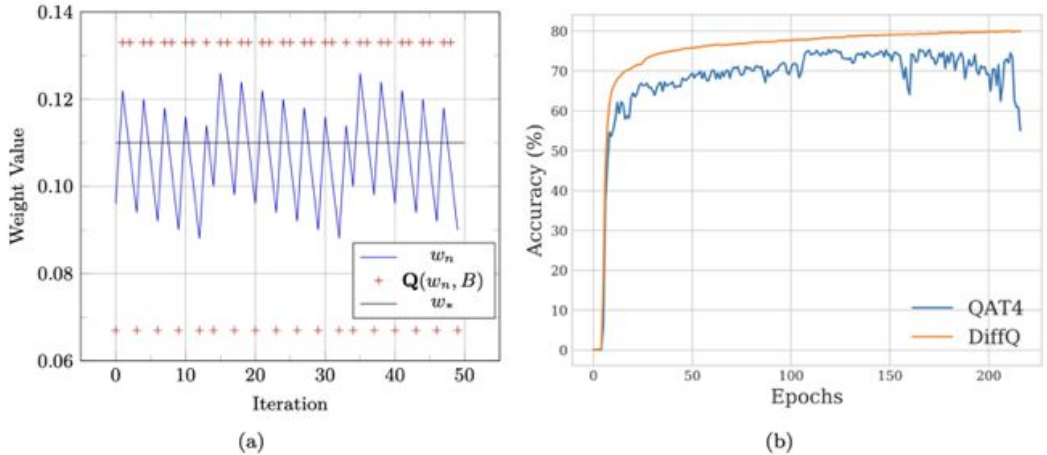


Figure 1: **(a)** Using STE and SGD to optimize the 1D least-mean-square problem given by equation 4 (with $B = 4$ and $X = 1$ a.s.). $\mathbf{Q}(w_n, B)$ oscillates between the quantized value just above (w_+) and just under (w_-) the unquantized ground truth w_*, while w_n oscillates around the boundary $(w_+ + w_-)/2$. **(b)** Model accuracy vs. epochs for ImageNet using EfficientNet-b3. Results are presented for both QAT over 4 bits and DIFFQ.

$$QW_b = Q(W, b)$$

$$QN = QW_b - W \approx \frac{\Delta_b}{2}\mathbf{U}[-1, 1]$$

$$Q\hat{W}_b = W + \frac{\Delta_b}{2}\mathbf{U}[-1, 1]$$

$$y = F(x, W + \frac{\Delta_b}{2}\mathbf{U}[-1, 1])$$

Differentiable Model Compression via Pseudo Quantization Noise, Alexandre Défossez et al, TMLR, 2022

Обучение в FP8: три матричных произведения

Для $Y = WX$ обозначим входящий градиент $G = \partial\mathcal{L}/\partial Y$:

$$\underbrace{Y = WX}_{\text{forward}}, \qquad \underbrace{\nabla_X \mathcal{L} = W^\top G}_{\text{backward по входу}}, \qquad \underbrace{\nabla_W \mathcal{L} = GX^\top}_{\text{backward по весам}}.$$

- В DeepSeek-V3 входы этих GEMM квантуются в FP8; выходы имеют формат BF16 или FP32.
- **Mixed precision:** дорогие умножения выполняются в низкой точности, чувствительные операции – в более высокой.
- Master weights и накопленные градиенты весов хранятся в FP32; моменты AdamW – в BF16.
- Нормализации, attention, embedding, выходная голова и MoE-router остаются в BF16/FP32.

Здесь FP8 ускоряет само обучение. Наличие fake quantization в QAT само по себе такого ускорения не даёт.

DeepSeek-V3 Technical Report, §3.3: FP8 mixed precision training.

DeepSeek-V3: как удержать точность FP8

- **E4M3 для входов GEMM**, включая градиенты: локальные масштабы позволяют выбрать больше бит мантиссы.
- В forward: активации – группы **128 каналов одного токена**; веса – блоки 128×128. Scales вычисляются по текущим данным.
- На H800 частичные суммы каждых 128 произведений переносятся в FP32-накопление на CUDA-ядрах.

$$Y_{ij} \approx \sum_{g} \underbrace{s^{W}_{i,g} s^{X}_{g,j}}_{\text{масштаб группы}} \underbrace{\sum_{k \in g} Q^{W}_{ik} Q^{X}_{kj}}_{\text{FP8-входы}} .$$

Здесь g разбивает размерность суммирования. Разные scales нельзя вынести одним множителем за всю сумму. В backward разбиение согласуется с ориентацией каждого GEMM.

Пример масштаба: если максимум группы 28, то $s = 28/448 = 1/16$. Значение 7 кодируется числом E4M3 112, а восстанавливается как $112/16 = 7$.

DeepSeek-V3 Technical Report, §3.3.2. Числовой пример и обозначения – учебные.

Пример шага обучения со смешанной точностью

Один линейный слой. $\mathrm{gemm8}$ подбирает scales, квантует входы в FP8 и возвращает BF16/FP32. Псевдокод:

```
grad_fp32 = 0
W = cast_bf16(W_master_fp32)
Y = gemm8(W, X)                    # forward
loss, G = loss_and_grad(Y, target)
dX = gemm8(transpose(W), G)        # backward по входу
dW = gemm8(G, transpose(X))        # backward по весам
grad_fp32 += cast_fp32(dW)
W_master_fp32 = optimizer_step(W_master_fp32, grad_fp32)
```

dX передаём предыдущему слою. После шага оптимизатора обнуляем градиент; FP8-представление весов готовим заново.

Зачем master weights: при $s = 1$ числа 1 и 0.999 имеют один E4M3-код. FP32 сохраняет малые обновления, которые со временем изменят FP8-код.

Учебный пример по схеме mixed precision из DeepSeek-V3; управление состоянием оптимизатора опущено.

OBQ: постановка задачи квантизации слоя

$W \in \mathbb{R}^{m \times d}$ – веса обученного слоя; $X \in \mathbb{R}^{d \times n}$ – калибровочные входы; $\mathcal{Q}_{ij}$ – заранее выбранные сетки.

$$\min_{\widehat{W}} \ \frac{1}{2}\|WX - \widehat{W}X\|_F^2 \quad \text{при} \quad \widehat{W}_{ij} \in \mathcal{Q}_{ij}.$$

- Сохраняем **выход слоя на данных**; не просто минимизируем расстояние между W и $\widehat{W}$.
- Точки сетки – восстановленные вещественные значения. Метки для калибровочных входов не нужны.
- Совместный дискретный поиск дорог. OBQ жадно фиксирует по одному весу, компенсируя ошибку оставшимися.

Используем идею OBS: вместо ограничения «вес равен нулю» вводим «вес равен выбранной точке сетки».

Frantar et al. Optimal Brain Compression, NeurIPS 2022: Optimal Brain Quantization (OBQ).

От потери сети к реконструкции слоя

Пусть $W \in \mathbb{R}^{m \times d}$, $X \in \mathbb{R}^{d \times n}$ – калибровочные входы.

$$\mathcal{E}(\widehat{W}) = \tfrac{1}{2}\|WX - \widehat{W}X\|_F^2 = \sum_{i=1}^{m} \tfrac{1}{2}\|(w_i - \widehat{w}_i)^\top X\|_2^2.$$

Минимизируем $\mathcal{E}$ по $\widehat{W} \in \mathcal{Q}$. Здесь w_i – транспонированная i-я строка; для неё

$$H = XX^\top, \qquad \tfrac{1}{2}\|\delta^\top X\|_2^2 = \tfrac{1}{2}\delta^\top H \delta.$$

- Это **точный гессиан квадратичной задачи слоя**, не гессиан всей LLM.
- Он одинаков для всех выходных строк: зависит только от X.
- Используем $H_\lambda = XX^\top + \lambda I$ для устойчивости.

Optimal Brain Compression, 2022. Множитель 1/2 выбран для удобства; в GPTQ-статье $H = 2XX^\top$.

OBQ: вместо нуля – значение на сетке

Хотим зафиксировать w_p в $q_p = \mathrm{quant}(w_p)$. Обозначим $r_p = w_p - q_p$:

$$e_p^\top \delta = q_p - w_p = -r_p.$$

Тот же вывод даёт

$$\boxed{\delta^* = -\frac{r_p}{C_{pp}} C_{:,p}}, \qquad \boxed{\Delta\mathcal{L}_p \approx \frac{r_p^2}{2C_{pp}}}.$$

- OBS: $q_p = 0$. Далее quant возвращает вещественное значение на сетке, то есть результат quantize + dequantize.
- Простое округление меняет один вес. OBQ дополнительно корректирует остальные.
- Выбираем p с минимальным $r_p^2/(2C_{pp})$ среди свободных весов; после фиксации исключаем его из коррекции.

Frantar et al. Optimal Brain Compression, NeurIPS 2022.

После фиксации веса: уменьшаем задачу

Пусть F – свободные координаты, $C = (H_{\lambda,FF})^{-1}$, $p \in F$.

$$\delta_F = -\frac{w_p - q_p}{C_{pp}} C_{:,p}.$$

Для следующих шагов нужен обратный гессиан на $F' = F \setminus \{p\}$:

$$\boxed{(H_{\lambda,F'F'})^{-1} = \left(C - \frac{C_{:,p} C_{p,:}}{C_{pp}}\right)_{F',F'}}.$$

- Это тождество блочного обращения, без повторной полной инверсии.
- Оно стоит $O(|F|^2)$ на шаг.
- В OBQ каждая строка может выбирать свой p: наборы F расходятся, и обновления C нельзя переиспользовать между строками.

Optimal Brain Compression, 2022. Обновление обратной матрицы при исключении координаты.

GPTQ: один порядок для всех строк

Выбираем общий порядок столбцов $1, \ldots, d$ и квантуем весь столбец сразу:

$$q = \mathrm{quant}(W_{:,p}), \quad r = W_{:,p} - q, \qquad \boxed{W_{:,F} \leftarrow W_{:,F} - r\frac{C_{p,F}}{C_{pp}}}.$$

- Свободный набор F теперь общий: хватает одной матрицы C.
- Обновление весов – внешнее произведение столбца ошибок и строки C.
- Общий порядок – эвристика; качество не гарантировано независимым от порядка.

$$\underbrace{O(md^3)}_{\text{OBQ, отдельные порядки}} \longrightarrow \underbrace{O(d^3 + md^2)}_{\text{общий порядок}}.$$

Frantar et al. GPTQ, ICLR 2023. Стоимость основной процедуры; сбор статистики X считается отдельно.

GPTQ: Холецкий вместо повторных downdate

Положим $C = H_\lambda^{-1} = R^\top R$, где R – верхнетреугольная матрица.

$$R = \begin{pmatrix} a & b^\top \\ 0 & D \end{pmatrix} \Rightarrow C = \begin{pmatrix} a^2 & ab^\top \\ ab & bb^\top + D^\top D \end{pmatrix}.$$

Исключение первой координаты оставляет $D^\top D$: вклад $bb^\top$ вычитается. Поэтому на шаге p:

$$C_{pp}^{(p)} = R_{pp}^2, \qquad C_{p,p:}^{(p)} = R_{pp} R_{p,p:}.$$

$$\boxed{e = \frac{W_{:,p} - q}{R_{pp}}, \qquad W_{:,p:} \leftarrow W_{:,p:} - e R_{p,p:}}.$$

Фактор R готовим один раз. В авторской реализации есть и вычисление H_λ^{-1} через Холецкого: полная инверсия не «исчезает», исчезают последовательные downdate.

GPTQ, 2023; авторская реализация gptq.py. $C^{(p)}$ – обратный гессиан текущих свободных координат.

GPTQ: накапливаем коррекции блоком

Для блока столбцов $B = [a, b)$ последовательно обрабатываем $p \in B$:

$$Q_{:,p} = \mathrm{quant}(W_{:,p}), \qquad E_{:,p-a} = \frac{W_{:,p} - Q_{:,p}}{R_{pp}},$$

$$W_{:,p:b} \leftarrow W_{:,p:b} - E_{:,p-a} R_{p,p:b}.$$

После блока обновляем оставшиеся столбцы одним матричным произведением:

$$\boxed{W_{:,b:d} \leftarrow W_{:,b:d} - E\, R_{a:b,b:d}}.$$

- Внутри блока применяем коррекции сразу; за его пределами откладываем.
- При фиксированной сетке будущие значения q не меняются от этой перестановки операций в точной арифметике.
- Число арифметических операций того же порядка; обращения к памяти организованы эффективнее.

GPTQ: авторская реализация. Интервалы p:b и a:b здесь полуоткрытые.

GPTQ: вся процедура

1. Собрать калибровочные входы X очередного линейного слоя.
2. Выбрать сетку, гранулярность и общий порядок столбцов.
3. Вычислить $H = XX^\top$, добавить λI; получить верхний фактор R матрицы H_λ^{-1}.
4. По блокам: округлить столбец, сохранить его в Q, компенсировать ошибку в ещё свободных весах.
5. Упаковать коды и scales; перейти к следующему слою / блоку модели.

Что получили: PTQ с данными калибровки и компенсацией ошибки, без backpropagation через всю LLM. GPTQ определяет способ выбора квантованных весов.

Формат упаковки и ядро для инференса – отдельные компоненты.

GPTQ, ICLR 2023: статья и открытая реализация IST-DASLab.

SparseGPT: возвращаемся к спарсификации

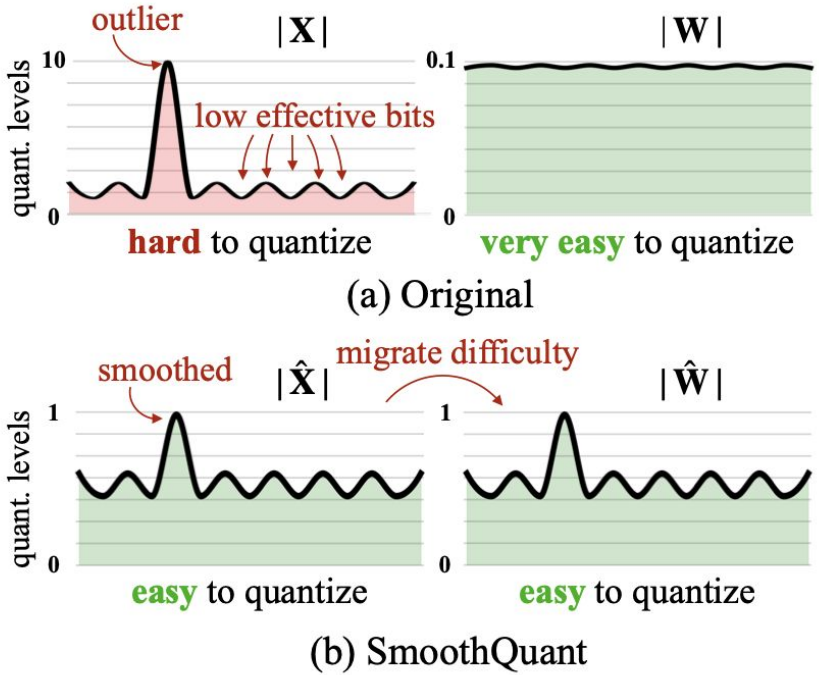
Теперь используем ту же послойную задачу и компенсацию, но целевые значения задаёт маска:

$$q_{ip} = \begin{cases} 0, & M_{ip} = 0, \\ W_{ip}, & M_{ip} = 1, \end{cases} \qquad e_i = \frac{W_{ip} - q_{ip}}{R_{pp}}.$$

- Для удаляемого веса оценка ошибки пропорциональна W_{ip}^2 / R_{pp}^2.
- Выбираем маску блоками и компенсируем удаление в ещё не обработанных столбцах.
- Даже сохранённые обработанные веса фиксируются: это позволяет разделять обратный гессиан между строками.
- Поддерживаются неструктурные и N:M-маски; возможно сочетание с квантизацией.

Frantar, Alistarh. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot, ICML 2023.

LLM Quantization: Outliers



SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models, Guangxuan Xiao et al, ICML, 2023

AWQ: защищаем важные каналы масштабом

В записи $Y = WX$ выберем положительную диагональную матрицу S:

$$WX = (WS)(S^{-1}X), \qquad \widehat{Y} = \mathrm{quant}(WS)(S^{-1}X).$$

- Важность каналов оцениваем по калибровочным активациям.
- Подбираем масштабы, чтобы уменьшить ошибку после квантизации весов.
- Важные веса не обязательно хранить в FP16: меняем параметризацию, сохраняя общий низкобитный формат.
- AWQ – weight-only PTQ; наличие активаций в калибровке не означает их квантизацию.

Lin et al. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration, MLSys 2024.

Вращения: распределить выброс по координатам

Пусть $U^\top U = I$. Тогда для линейного слоя

$$\boxed{WX = (WU)(U^\top X)}.$$

- До квантизации это то же преобразование.
- В новой системе координат большой отдельный компонент может распределиться по многим координатам.
- Меньше выбросов относительно типичных значений – проще квантовать.
- Нельзя произвольно протащить вращение через нелинейность: нужна согласованная перестройка графа.

Ashkboos et al. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs, NeurIPS 2024.

QuaRot и SpinQuant

QuaRot

- Структурированные случайные вращения, включая Hadamard.
- Часть преобразований сливается с весами; часть выполняется online.
- Квантизация весов, активаций и KV-cache до 4 бит в исследуемых моделях.

SpinQuant

- Разные вращения дают разную ошибку после квантизации.
- Вращения обучаются по калибровочным данным с ограничением ортогональности.
- Это дополнительная оптимизация преобразования перед экспортом.

Связь с QuIP#: общая идея подавления выбросов вращением; способы квантизации и преобразования модели различаются.

QuaRot, NeurIPS 2024; Liu et al. SpinQuant: LLM Quantization with Learned Rotations, ICLR 2025.

Какую задачу решает каждый метод?

Метод	Основная идея	Типичный объект
GPTQ	Компенсация ошибки весами	Веса
AWQ	Масштабы важных каналов	Веса
SmoothQuant	Перенос выбросов в веса	Веса + активации
QuaRot / SpinQuant	Вращения перед квантизацией	Веса + активации; KV-cache
QAT	Адаптация к целевой сетке	Зависит от fake-quant схемы

Методы не всегда взаимоисключающие: вращение можно сочетать с GPTQ; QAT может учитывать выбранные формат и гранулярность.

GPTQ; AWQ; SmoothQuant; QuaRot; QAT.

LLM Quantization: Outliers

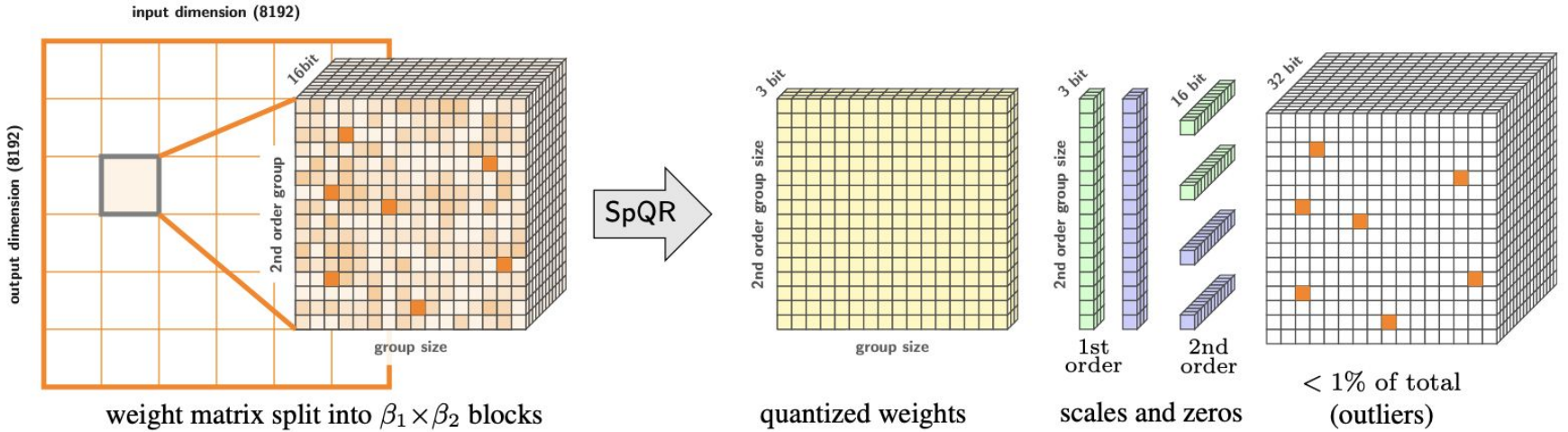


Figure 3: A high-level overview of the SpQR representation for a single weight tensor. The right side of the image depicts all stored data types and their dimensions.

SPQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression, Tim Dettmers et al, ICLR 2024

LLM Quantization: Scaling Laws

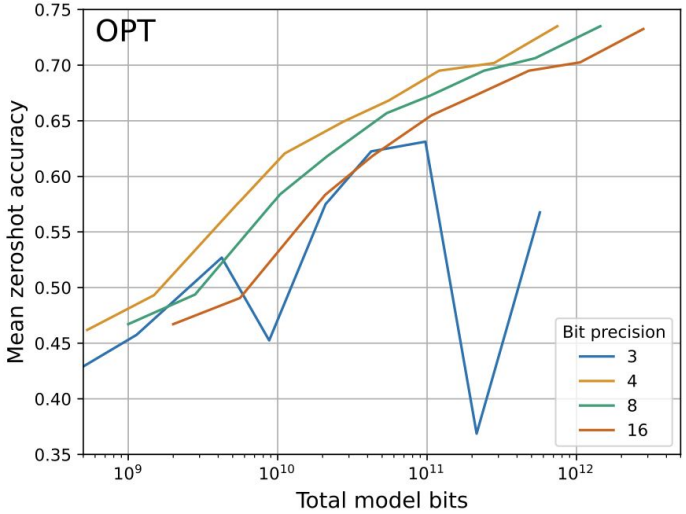


Figure 1. Bit-level scaling laws for mean zero-shot performance across four datasets for 125M to 176B parameter OPT models. Zero-shot performance increases steadily for fixed model bits as we reduce the quantization precision from 16 to 4 bits. At 3-bits, this relationship reverses, making 4-bit precision optimal.

We run more than 35,000 experiments with 16-bit inputs and k-bit parameters to examine which zero-shot quantization methods improve scaling for 3 to 8-bit precision at scales of 19M to 176B parameters across the LLM families BLOOM, OPT, NeoX/Pythia, and GPT-2. We find that it is challenging to improve the bit-level scaling trade-off, with the only improvements being the use of a small block size – splitting the parameters into small independently quantized blocks – and the quantization data type being used

The case for 4-bit precision: k-bit Inference Tim Dettmers, Luke Zettlemoyer, ICML, 2023

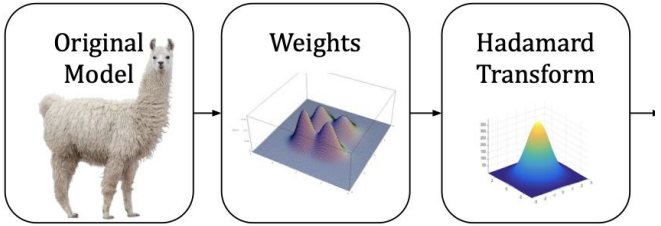
AQLM vs QUIP#

Algorithm 1 AQLM: Additive Quantization for LLMs

Require: model, data

$\mathbf{X}_{block} :=$ `model.input_embeddings(data)`
for $i = 1, \ldots,$ `model.num_layers` **do**
  `block := model.get_block(i)`
  $\mathbf{Y}_{block} :=$ `block`$(\mathbf{X}_{block})$
  for `layer` $\in$ `linear_layers(block)` **do**
    $\mathbf{W} :=$ `layer.weight`
    $\mathbf{X} :=$ `layer_inputs(layer,`$\mathbf{X}_{block}$`)`
    $C, b, s :=$ `initialize`$(\mathbf{W})$ `// k-means`
    while loss improves by at least τ **do**
      $C, s :=$ `train_Cs_adam`$(\mathbf{X}\mathbf{X}^T, \mathbf{W}, C, b, s)$
      $b :=$ `beam_search`$(\mathbf{X}\mathbf{X}^T, \mathbf{W}, C, b, s)$
    end while
    `/* save for fine-tuning */`
    `layer.weight :=` `AQLMFormat`(C, b, s)
  end for
  $\theta :=$ `trainable_parameters(block)`
  while loss improves by at least τ **do**
    $L := ||$`block`$(\mathbf{X}_{block}) - \mathbf{Y}_{block}||_2^2$
    $\theta :=$ `adam`$(\theta, \frac{\partial L}{\partial \theta})$
  end while
  $\mathbf{Y}_{block} :=$ `block`$(\mathbf{X}_{block})$
end for

январь 2024



Definition 1. *We say a symmetric Hessian matrix $H \in \mathbb{R}^{n\times n}$ is μ-incoherent if it has an eigendecomposition $H = Q\Lambda Q^T$ such that for all i and j, $|Q_{ij}| = \left|e_i^T Q e_j\right| \leq \mu/\sqrt{n}$. By extension, we say a weight matrix $W \in \mathbb{R}^{m\times n}$ is μ-incoherent if all i and j, $|W_{ij}| = \left|e_i^T W e_j\right| \leq \mu \|W\|_F / \sqrt{mn}$.*

Algorithm 3 QuIP: Quantization with Incoherence Processing

Require: $b \in \mathbb{N}$, $H \in \mathbb{R}^{n\times n}$ SPD, $W \in \mathbb{R}^{m\times n}$, $\mathcal{Q} \in \{\mathsf{Near}, \mathsf{Stoch}\}$, $\rho \in \mathbb{R}_+$, $\alpha \in [0, 1]$

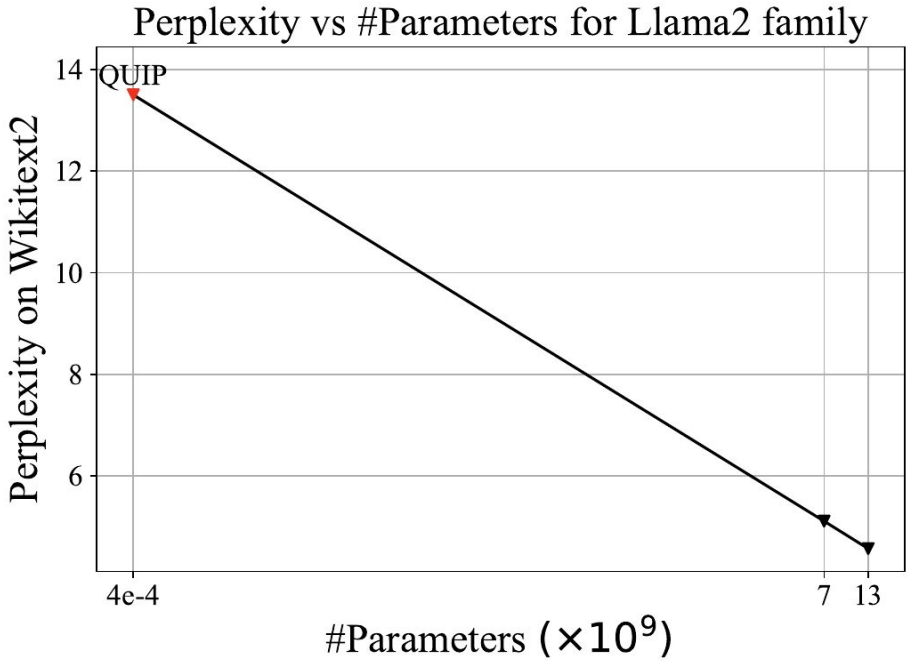
$\hat{W}, H, s, \tilde{D} \leftarrow \mathrm{Alg}\,1(b, H, W, \rho, \alpha)$ ▷ QuIP Incoherence Pre-Procesing
$H = (\dot{U} + I)D(\dot{U} + I)^{-1}$ ▷ LDL decomposition
for $k \in \{1, \ldots, n\}$ **do** $\hat{W}_k \leftarrow \mathrm{clamp}(\mathcal{Q}(W_k + (W - \hat{W})\dot{U}_k), 0, 2^b - 1)$ ▷ LDLQ
return $\hat{W} \leftarrow \mathrm{Alg}\,2(b, H, \hat{W}, s, \tilde{D})$ ▷ QuIP Incoherence Post-Processing

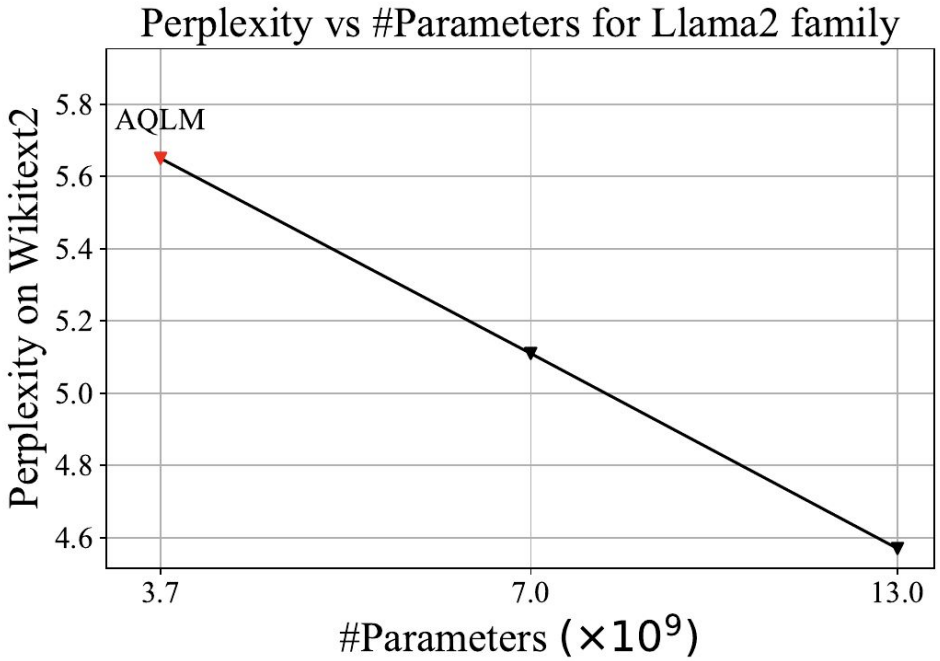
середина 2023

Extreme Compression of Large Language Models via Additive Quantization, Egiazarian et al, ICML, 2024

QUIP: 2-Bit Quantization of Large Language Models With Guarantees, Jerry Chee et al, NeurIPS, 2023

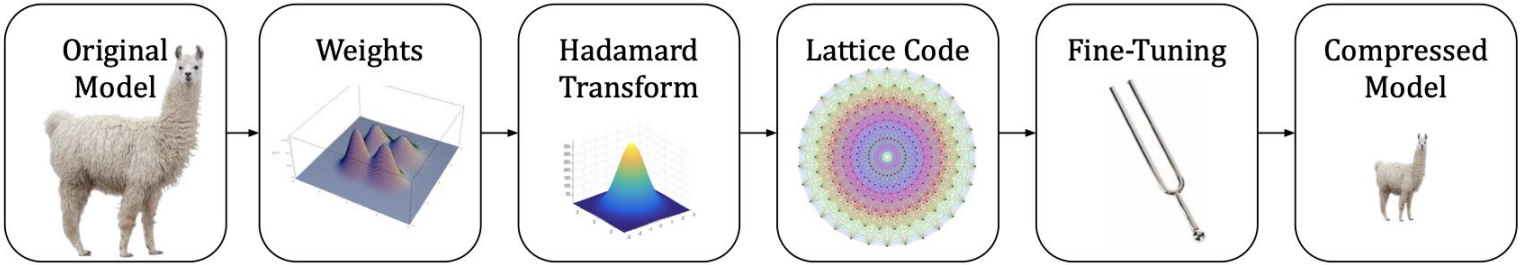
AQLM vs QUIP#



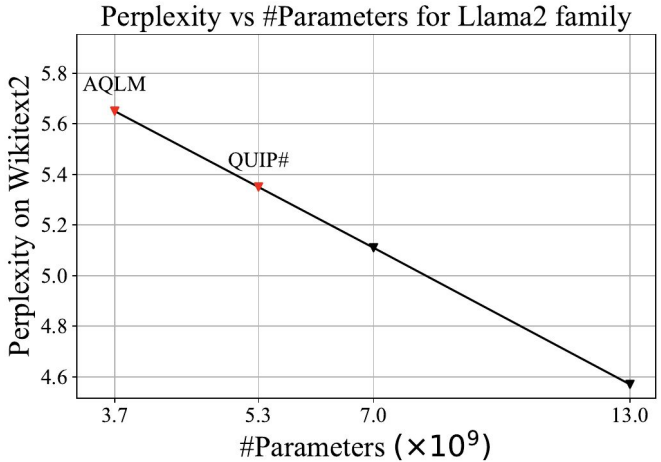


Extreme Compression of Large Language Models via Additive Quantization, Egiazarian et al, ICML, 2024
QUIP: 2-Bit Quantization of Large Language Models With Guarantees, Jerry Chee et al, NeurIPS, 2023

AQLM vs QUIP#



февраль 2024



QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks, Albert Tseng et al, ICML, 2024

AQLM vs QUIP#

Algorithm 1 PV algorithm

Initialization: starting point $x^0 \in \mathbb{R}^d_{\leq c}$
for $k = 0, 1, \ldots$ **do**
  $y^k = M_P(x^k) := \arg\min_{y \in \mathbb{R}^d} \{\phi(y) : P(y) \supseteq P(x^k)\}$ (P step: continuous)
  $x^{k+1} = M_V(y^k) := \arg\min_{x \in \mathbb{R}^d} \{\phi(x) : V(x) \subseteq V(y^k)\}$ (V step: discrete)
end for

Algorithm 2 PV-Tuning: Optimization

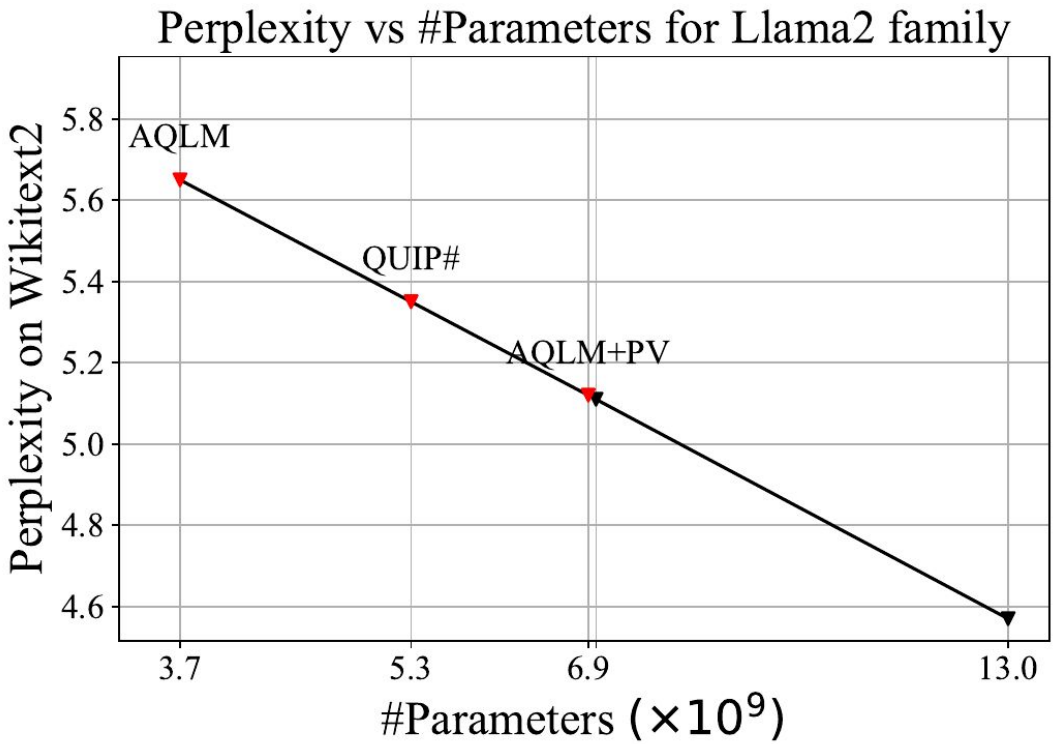
Require: initial parameters $x^0 \in \mathbb{R}^d_c$,
objective function $\phi : \mathbb{R}^d \to \mathbb{R}$,
subspace size $\tau \in [d]$

for $k = 0, \ldots, K-1$ **do**
  ▷ **P step:** update $V(x)$ by backprop
  $y^k = \arg\min_{y \in \mathbb{R}^d_{\leq c}} \{\phi(y) : P(y) \supseteq P(x^k)\}$
  ▷ **V step:** choose a subspace $\mathcal{S}^k$ & update $P(x)$
  $\mathcal{S}^k = \arg\operatorname{top}_{1 \leq i \leq d} \tau \ |\nabla_i \phi(y^k)|$ ▷ find τ largest
  $\widehat{\phi}_{y,\mathcal{S}^k}(x) := \left\| x - \left(y - \frac{1}{L_{\mathcal{S}^k}} Z^k \left(\nabla \phi(y) \right) \right) \right\|^2$
  $x^{k+1} = \arg\min_x \left\{ \widehat{\phi}_{y^k,\mathcal{S}^k}(x) : V(x) \subseteq V(y^k) \right\}$
end for

май 2024

PV-tuning: Beyond Straight-Through Estimation for Extreme LLM Compression, Vladimir Malinovskii et al, NeurIPS, 2024

AQLM vs QUIP#


Perplexity vs #Parameters for Llama2 family
Perplexity on Wikitext2
5.8
5.6
5.4
5.2
5.0
4.8
4.6
AQLM
QUIP#
AQLM+PV
3.7
5.3
6.9
13.0
#Parameters (×10⁹)

AQLM vs QUIP#

AQLM

Dettmers & Zettlemoyer (2022) examined the accuracy-compression trade-offs of these early methods, suggesting that 4-bit quantization may be optimal for RTN quantization, and observing that data-aware methods like GPTQ allow for higher compression, i.e. strictly below 4 bits/weight, maintaining Pareto optimality. Our work brings this Pareto frontier below 3 bits/weight, for the first time. Parallel

AQLM+PV

Pareto-optimality. A key practical question concerns obtaining optimal quality for the target model size, where a smaller model compressed to 3-4 bits often dominates a larger model compressed to 1-bit. The best known Pareto-optimal bit-width for Llama 2 is 2.5 [19]: compressing a larger model to less than 2.5 bits per weight is inferior to a smaller model quantized to the same total number of bytes. From this perspective, **PV-tuning pushes the Pareto-optimal frontier for LLAMA 2 to 2.0 bits**. This is easiest to see in Table 8: a 2-bit 13B model outperforms any 7B quantization and is comparable with the 16-bit 7B model. The same holds for the 2-bit 70B model.

KV-cache quantization

The core idea of AQUA-KV is to leverage inter-dependencies between consecutive KV-caches to improve compression. **For this, we train compact predictors that "guess" the value of a Key & Value pair using other cache entries, then quantize the residual information that could not be predicted.** This way, we only store the information that cannot be recovered from other sources.

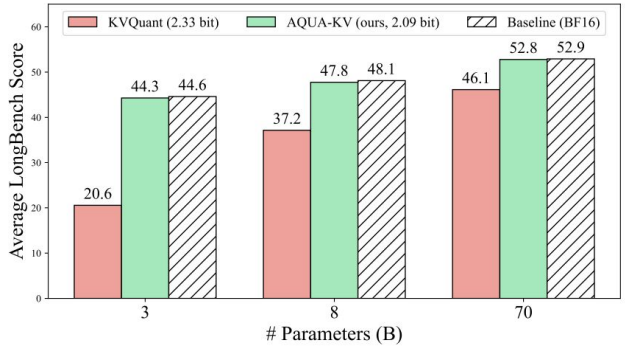


Figure 1. Comparison of AQUA-KV to alternative Key-Value Cache compression methods for Llama 3.x models in terms of average LongBench score on 14 english tasks (see Section 4).

Algorithm 1 AQUA-KV Calibration

Require: model, data, quantization method Q

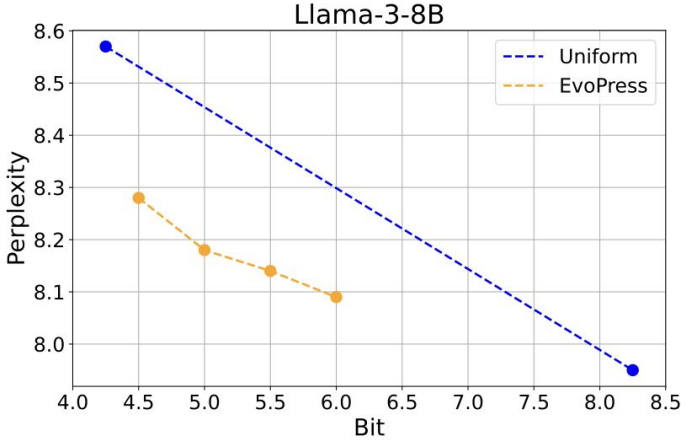
$\mathbf{X} \leftarrow$ `model.input_embeddings(data)`
$\mathbf{K}_{\text{old}}, \mathbf{V}_{\text{old}} \leftarrow \emptyset$
`predictors` $\leftarrow \{\}$
for $i = 1, \ldots,$ `model.num_layers` **do**
  `block` $\leftarrow$ `model.transformer_layers`$[i]$
  $(\mathbf{K}, \mathbf{V}) \leftarrow$ `block.get_attention_kv`$(\mathbf{X})$
  $\mathbf{X} \leftarrow$ `block`$(\mathbf{X})$
  if $\mathbf{K}_{\text{old}} = \emptyset$ **and** $\mathbf{V}_{\text{old}} = \emptyset$ **then**
    $\mathbf{K}_{\text{old}}, \mathbf{V}_{\text{old}} \leftarrow \mathbf{K}, \mathbf{V}$
    continue
  end if
  # Predict keys from past keys
  $f_{\text{key}} \leftarrow \arg\min_f \|f(\mathbf{K}_{\text{old}}) - \mathbf{K}\|_2^2$
  $\mathbf{K}_{\text{residue}} \leftarrow Q^{-1}(Q(\mathbf{K} - f_{\text{key}}(\mathbf{K}_{\text{old}})))$
  $\mathbf{K}_{\text{rec}} \leftarrow f_{\text{key}}(\mathbf{K}_{\text{old}}) + \mathbf{K}_{\text{residue}}$
  # Predict values from past values and current keys
  $f_{\text{value}} \leftarrow \arg\min_f \|f([\mathbf{V}_{\text{old}}; \mathbf{K}_{\text{rec}}]) - \mathbf{V}\|_2^2$
  $\mathbf{V}_{\text{residue}} \leftarrow Q^{-1}(Q(\mathbf{V} - f_{\text{value}}([\mathbf{V}_{\text{old}}; \mathbf{K}_{\text{rec}}])))$
  $\mathbf{V}_{\text{rec}} \leftarrow f_{\text{value}}([\mathbf{V}_{\text{old}}; \mathbf{K}_{\text{rec}}]) + \mathbf{V}_{\text{residue}}$
  `predictors[i]` $= (f_{\text{key}}, f_{\text{value}})$
  $\mathbf{K}_{\text{old}}, \mathbf{V}_{\text{old}} \leftarrow \mathbf{K}_{\text{rec}}, \mathbf{V}_{\text{rec}}$
end for
return `predictors`

Cache Me If You Must: Adaptive Key-Value Quantization for Large Language Models, Alina Shutova et al, ICML, 2025

Квантизация в смешанную битность

$$\hat{M}^* = \arg\max_{\hat{M}} f(\hat{M}) \quad \text{subject to} \quad g(\hat{M}) \leq C,$$

where $f(\hat{M})$ quantifies the performance of the compressed model $\hat{M}$ and $g(\hat{M})$ represents the compression constraint. For simplicity, we will define g as the model's total size (in terms of parameters); however, the method can be adapted to other constraints, such as inference speed.



Algorithm 1: EvoPress: A $(1+\lambda)$-Evolutionary Algorithm with Level-Switch Mutation and Multi-Step Selection for Maximizing $f : [m]^n \to \mathbb{R}$.

Initialization: `candidates` $\leftarrow []$;
for $i \leftarrow 1$ **to** *initialCandidates* **do**
  `candidate` $\leftarrow$ `sampleUniformly()`;
  `candidates.append(candidate)`;
$x^{(1)} \leftarrow$ `selectTopKFittest(candidates, initialTokens,` $K = 1$`)`;
Optimization: **for** $t \leftarrow 1$ **to** ∞ **do**
  `offspring` $\leftarrow []$;
  Mutation: **for** $i \leftarrow 1$ **to** λ **do**
    $y_i \leftarrow x^{(t)}$;
    $y_i \leftarrow$ `LevelSwitchMutation`(y_i);
    `offspring.append`(y_i);
  Selection: **for** *step* $\leftarrow 1$ **to** *selectSteps* **do**
    Elitism: **if** *step* $=$ *selectSteps* **then**
      `offspring.append`$(x^{(t)})$;
    `offs.` $\leftarrow$ `selectTopKFittest(offs., tokens[step],` $K =$ `survivors[step])`;
  $x^{(t+1)} \leftarrow$ `offspring`$[0]$;

EvoPress: Towards Optimal Dynamic Model Compression via Evolutionary Search, Oliver Sieberling et al, ICML, 2025

Стабилизация обучения в низкой битности

1) Делаем QAT
2) Возьмем SOTA приемы из обычной квантизации (Hadamar Transform, подбор скейлинга, etc)
3) Нужно улучшить оценку градиентов
4) Давайте оставим только градиенты от параметров, на которых ошибка квантизации маленькая!
5) В какой-то степени эквивалентно обучению с сжатыми градиентами, такое оптимизация ест

Then, the squared gradient difference in (2) decomposes as:

$$\underbrace{\sum_{k \in S_{\text{small}}} (\nabla_{\mathbf{w}} L_k - \nabla_{\widehat{\mathbf{w}}} L_k)^2}_{(\star)} + \underbrace{\sum_{k \in S_{\text{large}}} (\nabla_{\mathbf{w}} L_k - \nabla_{\widehat{\mathbf{w}}} L_k)^2}_{(\star\star)}.$$

Assuming that the loss L is γ-smooth, the $(\star)$ "small error" term would be upper bounded by $\gamma^2 T^2 |S_{\text{small}}|$. Intu-

To bound the second term $(\star\star)$, we choose to *not trust* the gradient estimations for weights with large errors $\{\nabla_{\widehat{\mathbf{w}}} L_k : k \in S_{\text{large}}\}$. Choosing $T = \frac{\alpha^*}{2^b - 1}$ and masking gradients for elements in S_{large} we obtain the gradient operator:

$$\frac{\partial}{\partial \mathbf{x}} \approx \mathbf{I}_{|\hat{\mathbf{x}} - \mathbf{x}| \leq T} \odot \frac{\partial}{\partial \hat{\mathbf{x}}} := M_{\alpha^*}(\mathbf{x}; \hat{\mathbf{x}}) \odot \frac{\partial}{\partial \hat{\mathbf{x}}},$$

where $\mathbf{I}_{|\hat{\mathbf{x}} - \mathbf{x}| \leq T}$ is the standard indicator operator. We will refer to M_{α^*} as the "trust mask"; this gradient estimation operator will be called the **trust estimator**.

QuEST Stable Training of LLMs with 1-Bit Weights and Activations, Andrei Panferov et al, ICML, 2025

Стабилизация обучения в низкой битности

Algorithm 1 `QuEST Training Forward`

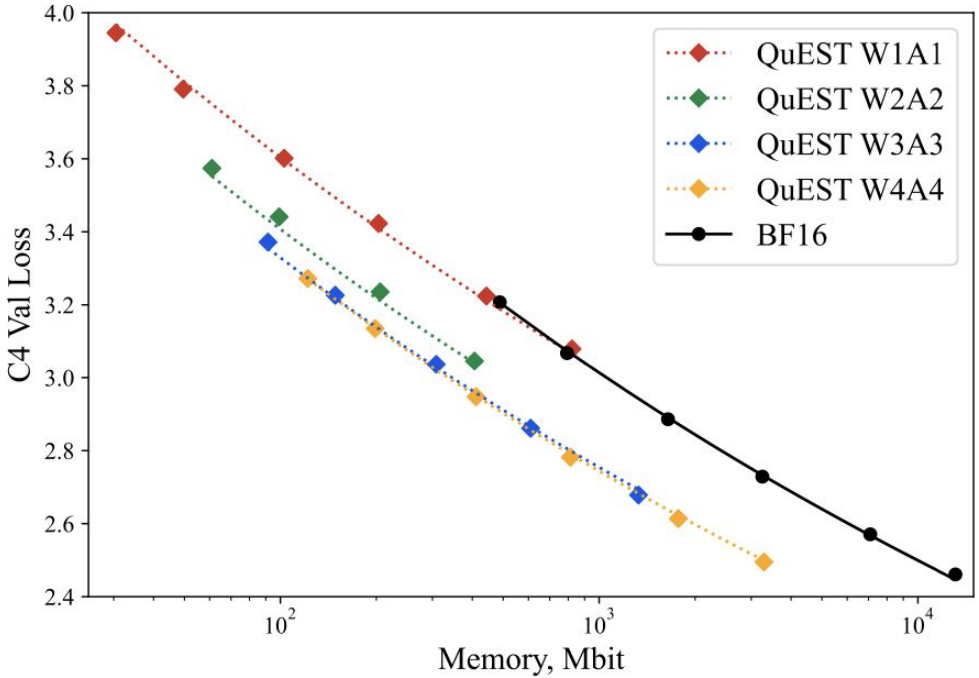
Input: Input activations $\mathbf{x}$, row-major weight $\mathbf{w}$
$\mathbf{x}_h = \mathrm{HT}(\mathbf{x})$
$\hat{\mathbf{x}}_h = \mathrm{proj}_{\alpha^*}\, \mathbf{x}_h$
$\mathbf{w}_h = \mathrm{HT}(\mathbf{w})$
$\hat{\mathbf{w}}_h = \mathrm{proj}_{\alpha^*}\, \mathbf{w}_h$
$\mathbf{y} = \hat{\mathbf{x}}_h \hat{\mathbf{w}}_h^T$
Return: $\mathbf{y}, \hat{\mathbf{x}}_h, \hat{\mathbf{w}}_h, M_{\alpha^*}(\mathbf{x}_h; \hat{\mathbf{x}}_h), M_{\alpha^*}(\mathbf{w}_h; \hat{\mathbf{w}}_h)$

Algorithm 2 `QuEST Training Backward`

Input: $\frac{\partial L}{\partial \mathbf{y}}, \hat{\mathbf{x}}_h, \hat{\mathbf{w}}_h, M_{\alpha^*}(\mathbf{x}_h; \hat{\mathbf{x}}_h), M_{\alpha^*}(\mathbf{w}_h; \hat{\mathbf{w}}_h)$
$\frac{\partial L}{\partial \hat{\mathbf{x}}_h} = \frac{\partial L}{\partial \mathbf{y}} \hat{\mathbf{w}}_h$
$\frac{\partial L}{\partial \mathbf{x}} = \mathrm{IHT}\left(M_{\alpha^*}(\mathbf{x}_h; \hat{\mathbf{x}}_h) \odot \frac{\partial L}{\partial \hat{\mathbf{x}}_h} \right)$
$\frac{\partial L}{\partial \hat{\mathbf{w}}_h} = \hat{\mathbf{x}}_h^T \frac{\partial L}{\partial \mathbf{y}}$
$\frac{\partial L}{\partial \mathbf{w}} = \mathrm{IHT}\left(M_{\alpha^*}(\mathbf{w}_h; \hat{\mathbf{w}}_h) \odot \frac{\partial L}{\partial \hat{\mathbf{w}}_h} \right)$
Return: $\frac{\partial L}{\partial \mathbf{x}}, \frac{\partial L}{\partial \mathbf{w}}$

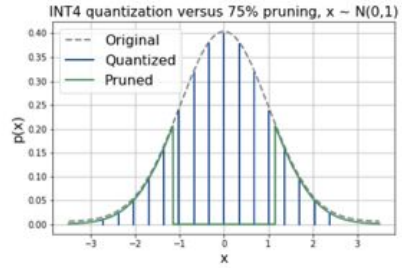
QuEST Stable Training of LLMs with 1-Bit Weights and Activations, Andrei Panferov et al, ICML, 2025

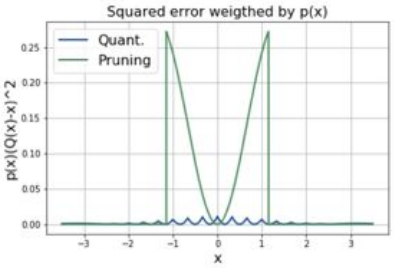
Стабилизация обучения в низкой битности

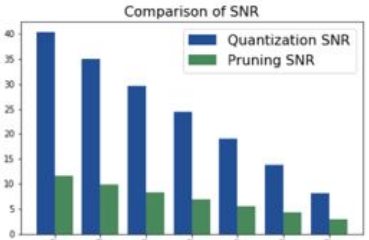


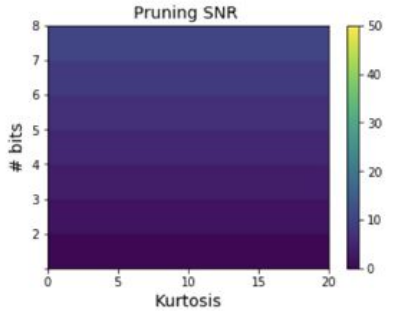
QuEST Stable Training of LLMs with 1-Bit Weights and Activations, Andrei Panferov et al, ICML, 2025

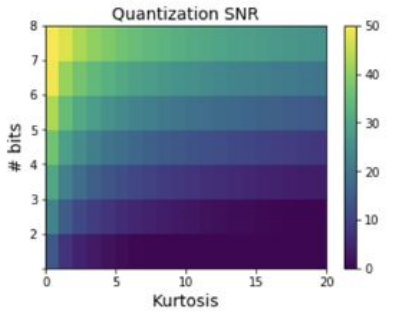
Quantization vs Pruning

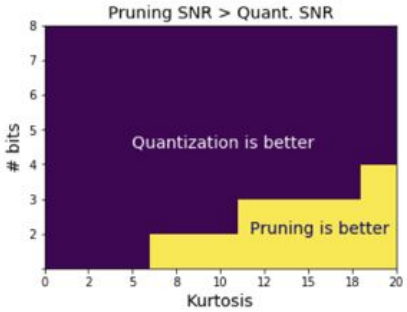












Pruning vs Quantization: Which is Better?, Kuzmin et al, Neurips, 2023

Quantization vs Pruning

$$\min_{\boldsymbol{w}} E(\boldsymbol{w}) = \|\boldsymbol{X}\delta\boldsymbol{w} - \boldsymbol{X}\boldsymbol{w}_{orig}\|_2^2$$
$$\text{s.t. } \boldsymbol{w} \in \mathbb{Z}^n,$$
$$w_{min} \leq w_i \leq w_{max},$$

$$\tilde{E}(\boldsymbol{w}) = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{P}\boldsymbol{w} - \boldsymbol{q}^T\boldsymbol{w},$$
$$\text{s.t. } \boldsymbol{w} \in \mathbb{Z}^n,$$
$$w_{min} \leq w_i \leq w_{max},$$

$$L(\boldsymbol{\lambda}) = \max -\gamma,$$
$$\text{s.t. } \begin{bmatrix} \boldsymbol{P} - \mathbf{diag}(\boldsymbol{\lambda}) & q + \frac{1}{2}\lambda \\ \left(q + \frac{1}{2}\lambda\right)^T & \gamma \end{bmatrix} \succeq 0,$$
$$\boldsymbol{\lambda} \geq 0,$$

$$E = \min_{\hat{\boldsymbol{w}}} \|\boldsymbol{X}\hat{\boldsymbol{w}} - \boldsymbol{X}\boldsymbol{w}_{orig}\|_2^2$$
$$\text{s.t. } \|\hat{\boldsymbol{w}}\|_0 \leq s,$$

$$E(\boldsymbol{w}) = \min_{\boldsymbol{w},\boldsymbol{m}} \|\boldsymbol{X}(\boldsymbol{m} \odot \boldsymbol{w}) - \boldsymbol{X}\boldsymbol{w}_{orig}\|_2^2,$$
$$\text{s.t. } \|\boldsymbol{m}\|_1 = s,$$
$$-\boldsymbol{m} \odot l \leq \hat{\boldsymbol{w}} \leq \boldsymbol{m} \odot u$$
$$l, u > 0, m_i \in \{0, 1\},$$

Pruning vs Quantization: Which is Better?, Kuzmin et al, Neurips, 2023

Quantization vs Pruning

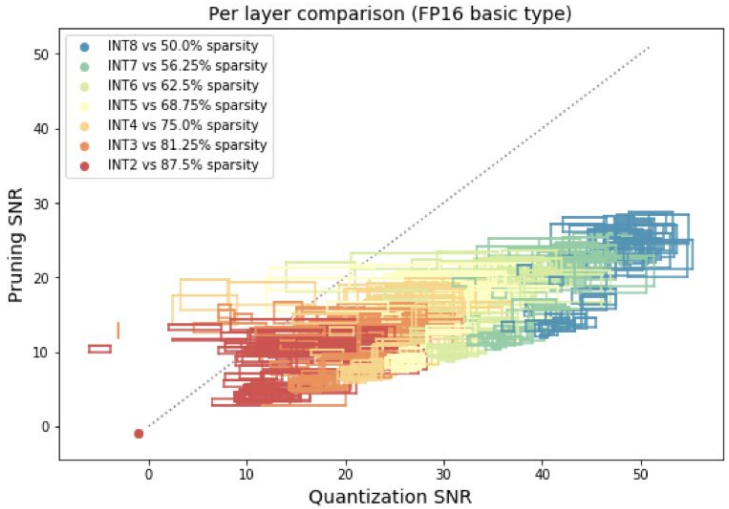


Figure 4: Comparison in the post-training scenario. Each box corresponds to a subset of one of 10 layers from the 4 different models that were used, with 7 different bit-width comparison points. The ranges of the box indicate the lower and higher-bounds found by the algorithms.

Pruning vs Quantization: Which is Better?, Kuzmin et al, Neurips, 2023

Quantization vs Pruning

Model	Orig.	Metric	Method	8b	7b	6b	5b	4b	3b	2b
Resnet-18	69.7	acc.	quant.	**70.5**	**70.5**	**70.6**	**70.3**	**70.0**	**68.9**	**67.3**
			pruning	70.3	70.1	69.9	69.5	69.3	68.3	66.8
Resnet-50	76.1	acc.	quant.	76.4	**76.4**	**76.4**	**76.3**	**76.2**	**75.5**	72.3
			pruning	**76.6**	**76.4**	76.2	76.1	75.9	75.4	**74.3**
MobileNet-V2	71.7	acc.	quant.	**71.9**	**72.0**	**71.7**	**71.6**	**70.9**	**68.6**	**59.1**
			pruning	68.1	65.6	61.9	56.3	48.0	34.0	21.2
EfficientNet	75.4	acc.	quant.	**75.2**	**75.3**	**75.0**	**74.6**	**74.0**	**71.5**	**60.9**
			pruning	72.5	70.9	68.1	63.6	56.4	44.5	27.1
MobileNet-V3	67.4	acc.	quant.	**67.7**	**67.6**	**67.1**	**66.3**	**64.7**	**60.8**	**50.5**
			pruning	65.6	64.4	62.4	60.2	56.1	31.7	0.0
ViT	81.3	acc.	quant.	**81.5**	**81.4**	**81.4**	**81.0**	**80.4**	**78.4**	**72.2**
			pruning	76.6	76.6	76.2	73.1	72.4	71.5	69.4
DeepLab-V3	72.9	mIoU	quant.	**72.3**	**72.3**	**72.4**	**71.9**	**70.8**	**63.2**	**17.6**
			pruning	65.2	62.8	56.8	47.7	32.9	18.6	10.0
EfficientDet	40.2	mAP	quant.	**39.6**	**39.6**	**39.6**	**39.2**	**37.8**	**33.5**	**15.5**
			pruning	34.5	33.0	30.9	27.9	24.2	17.9	8.0
OPT-350m	14.8	perpl.	quant.	**14.8**	**14.8**	**14.9**	**15.0**	**15.3**	**15.9**	**19.9**
			pruning	18.0	19.7	22.6	27.2	35.4	53.5	101.4

Table 1: Comparison of QAT and magnitude pruning with fine-tuning given equal model size and equal number of epochs for fine-tuning.

Pruning vs Quantization: Which is Better?, Kuzmin et al, Neurips, 2023

Quantization vs Pruning

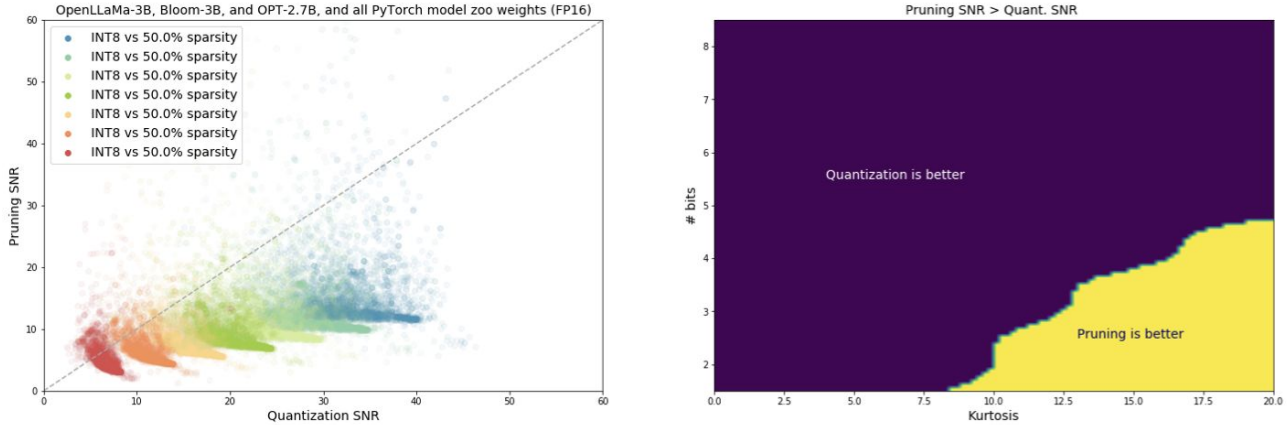


Figure 3: (left) Comparison on all the weights from PyTorch model zoo (46 models) combined with 3 large language models (Bloom-3b, Llama-3b, OPT-2.7b). (left) Pruning SNR versus quantization SNR for every tensor. (right) Pruning is preferable at high compression ratios for tensors with high sample kurtosis values.

Pruning vs Quantization: Which is Better?, Kuzmin et al, Neurips, 2023